

PREN Delta Robot Controller — Dokumentation

Pascal Hofstetter, Roman Winiger

2026

Table of Contents

Übersicht	1
Team	1
Softwarestand	1
Systemübersicht	1
Softwarearchitektur	3
Modulverantwortlichkeiten	3
Globale Datenstrukturen	4
<code>current_config_t</code> (globals.h)	4
Ramp-Arrays	4
Kompilierzeit-Konfigurationsflags	5
Startsequenz	5
Hardware	7
Mikrocontroller	7
Pin-Belegung	7
Schrittmotoren (STEP-Pins)	7
Richtungs-Pins (DIR)	7
Enable-Pins (EN)	8
Sensoren (Endschalter)	8
Spule (Elektromagnet)	8
Debug- / Reserve-Pins	8
Timer — FTM0	9
Timer-Takt und Auflösung	9
Schrittsignal-Timing	9
FTM0-Kanal-Zuordnung	9
NVIC-Prioritäten	10
Kommunikation	11
UART-Schnittstellen	11
Kommunikationsprotokoll (UART2)	11
Befehlssatz (Textkommunikation)	11
Terminal-Handler (<code>term.c</code>)	12
Fehlerbehandlung	12
Motorsteuerung	13
FTM0 Output-Compare — Grundprinzip	13
Kanalkonfiguration	13
CnV-Inkrement-Prinzip	13
Timer-Parameter	14
ISR-Dispatcher	14
Schrittpuls-Erzeugung (Normalbetrieb)	15

Zwei-Zustands-Automat (CH1/CH2/CH4)	15
Konkretes Timing-Diagramm	16
Rotationsmotor (CH5)	16
Overflow-Behandlung (OVERFLOW_HANDLING)	16
Problem	16
Auflösung in calcPulsePause()	16
Abarbeitung in der ISR	17
Initiale Pause (halbe Pause)	17
Step-Corrector (Rundungsfehler-Kompensation)	17
Ursache des Problems	17
Algorithmus	18
Anwendung in der ISR (am Ende des Puls-Ende-Pfads)	18
moveWay() — Vollständiger Ablauf	19
Ramp_Disabled -Parameter — Semantik	20
N-Schritt-Rampenprofil (RAMP_MODE_NSTEP)	20
Konzept: Zeitsegmente mit sinkender Dauer	20
Segment-Analyse: analyzeSegment() und calcEndPoint()	21
Segment-Übergang: Rem_Pending	21
CH6-ISR: Ramp-Schritt-Inkrementierer	22
Motor-ISR im Ramp-Modus (vereinfacht für CH2)	23
End-Rampe (RAMP_MODE_END)	24
Konzept	24
ftm0ReducePS() — Atomarer Prescaler-Wechsel	25
Referenzfahrt (moveToInitPos)	26
Dreiphasiger Ablauf	26
Sensoren	27
Synchronisation der drei Achsen	27
Master-Slave-Prinzip	27
Zahlenbeispiel	27
Emergency Stop (FTM0 CH7)	28
Konfiguration	28
CH7-ISR — Ablauf bei Auslösung	28
Globale Variablen — Lebenszyklus	29
initGlobalsMove() und resetMoveTimers()	29
Erstellen & Flashen	31
Voraussetzungen	31
Erstellen mit MCUXpresso	31
Erstellen über die Kommandozeile	31
Flashen & Debuggen	31
Kompilierzeit-Konfiguration anpassen	32
Debug-Modi	32

Übersicht

Dieses Projekt implementiert die eingebettete Firmware für einen **Delta-Roboter**, der im Rahmen des PREN-Moduls (Praktisches Projekt in Elektronik/Nachrichtentechnik) an der **Hochschule Luzern T&A**, Gruppe 1, 2026 entwickelt wurde.

Die Firmware läuft auf einem **NXP Kinetis K22 (MK22F51212)** Mikrocontroller und ist verantwortlich für:

- Empfang von Bewegungsbefehlen von einem Raspberry Pi über UART2 (strukturiertes Binärprotokoll)
- Ansteuerung von drei linearen Schrittmotoren (Delta-Achsen) mit präziser, synchronisierter Impulserzeugung
- Optionale Beschleunigungsrampe (NSTEP-Profil, 10 Stufen) beim Anfahren und Bremsen
- Steuerung eines Rotationsmotors am Endeffektor (Greifer-Rotation)
- Aktivierung eines Elektromagneten (Spule) zur Objektmanipulation (Pick & Place)
- Auslesen von Endschalter-Sensoren für die Referenzfahrt (Homing)
- Hardware-seitige Notabschaltung aller Motoren über FTM0-Kanal 7 (Emergency Stop)

Team

Name	Rolle
Pascal Hofstetter	Firmware-Entwicklung
Roman Winiger	Firmware-Entwicklung

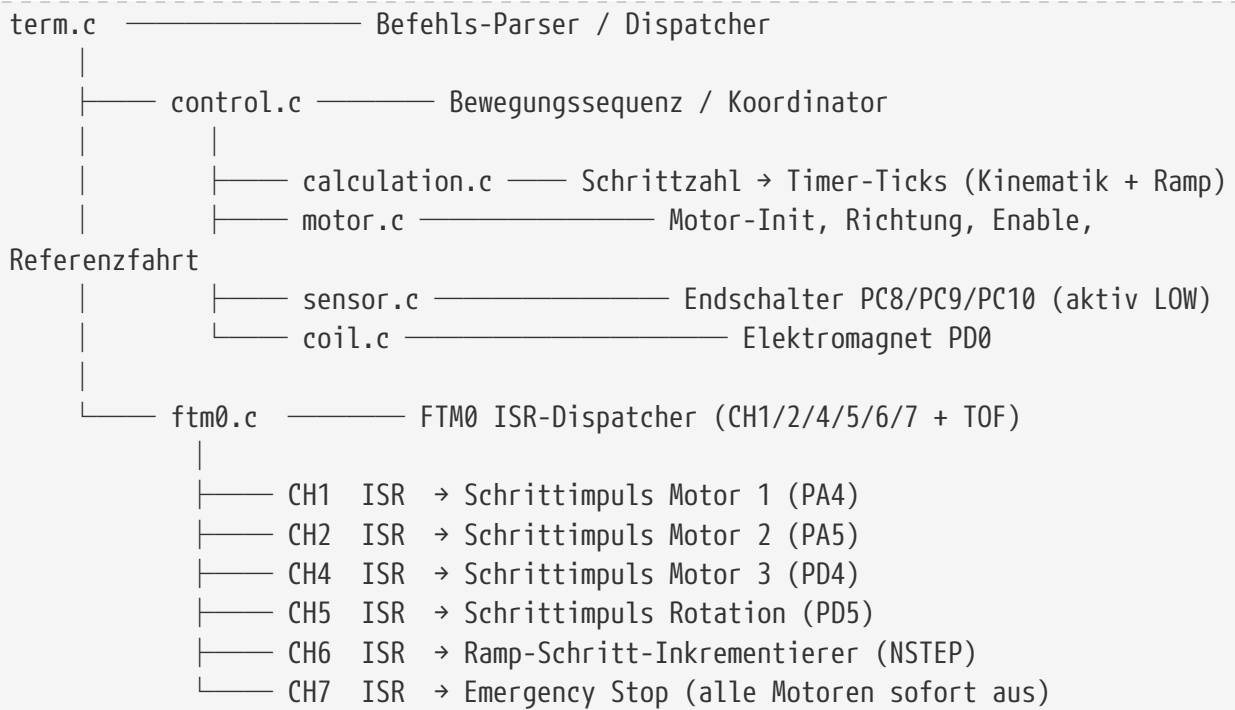
Softwarestand

Eigenschaft	Wert
Letzter Commit	5ba47ef — EndRamp Testing
Datum	2026-06-02
Branch	main
Aktive Rampenmodi	RAMP_MODE_NSTEP, RAMP_MODE_END
Rampen-Schritte	10 Stufen, je 20000 Ticks, +100 % pro Stufe

Systemübersicht

Die folgende Abbildung zeigt den Datenfluss auf hoher Ebene:

```
Raspberry Pi
|  UART2 (115200 Baud)
|  □
```



Softwarearchitektur

Die Firmware ist als Bare-Metal-C (kein RTOS) mit einem modularen Schichtendesign aufgebaut.

source/	
— main.c	Einstiegspunkt, Testsequenzen
— platform.h	Takt-Konfigurationen, Fehlercodes, Debug-Makros
— globals.h	Globaler Motorzustand, Timer-Parameter, Ramp-Arrays
— control/	Roboter-Steuerlogik
— control.c/h	Zentraler Befehls-Dispatcher, globale Variablen-Init
— motor/	
— motor.c/h	Motor-Init, Richtung, Enable, Referenzfahrt
— calculation.c/h	Schritt-Pause-Berechnung, Ramp-Profil-Berechnung
— motor_config.h	Pin-Makros (STEP/DIR/EN) für alle 4 Motoren
— emergency.c/h	Emergency-Stop-Init (FTM0 CH7)
— sensor/	
— sensor.c/h	Endschalter-Abfrage
— sensor_config.h	Pin-Makros für Sensoren PC8/PC9/PC10
— coil/	
— coil.c/h	Elektromagnet Ein/Aus
— coil_config.h	Pin-Makros für Spule PD0
— com/	Kommunikation
— uart.c/h	UART2-Treiber (Raspberry Pi, 115200 Baud)
— term.c/h	Befehls-Registry und -Dispatcher
— utils/	Hilfsfunktionen
— ftm0.c/h	FTM0-Init, ISR-Dispatcher, PS-Reduktion
— wait.c/h	Busy-Wait-Delays (µs/ms)
— debug.c/h	Debug-Hilfsfunktionen

Modulverantwortlichkeiten

Modul	Verantwortlichkeit
<code>control.c</code>	Initialisiert alle Subsysteme (Takt-Gating, GPIO, UART, FTM0, Sensoren, Spule); hält globale Motorzustandsvariablen; leitet empfangene Befehle an Motor-, Spulen- und Sensormodule weiter
<code>motor.c</code>	Konfiguriert FTM0-Kanäle als Output-Compare; setzt Richtungs- und Enable-Pins; führt die Referenzfahrt (<code>moveToInitPos</code>) durch; startet und stoppt die Impuls-ISRs
<code>calculation.c</code>	Berechnet aus den Schrittzahlen aller drei Achsen die zeitlich synchronisierten Pausen-Ticks; füllt alle Ramp-Arrays (<code>Ramp_Mx_Pause_Ticks</code> , <code>Ramp_Mx_Pulse_Ticks</code> , <code>Ramp_Mx_End_Rem_Ticks</code> und deren <code>_OF</code> -Varianten) vor dem Bewegungsstart; berechnet den Step-Corrector für Rundungsfehler

Modul	Verantwortlichkeit
<code>ftm0.c</code>	Implementiert <code>FTM0_IRQHandler</code> als Dispatcher für CH0–CH7 und TOF; enthält alle per-Kanal ISR-Routinen (CH1/2/4/5/6/7); stellt Hilfsfunktionen für Init, Clock-Start/-Stopp und Prescaler-Änderung bereit
<code>emergency.c</code>	Richtet FTM0 CH7 als Output-Compare-Interrupt ein; die CH7-ISR setzt alle Motor-Ausgänge auf LOW und deaktiviert alle Kanal-Interrupts dauerhaft
<code>sensor.c</code>	Liest die Endschalter-Zustände an PC8, PC9, PC10 per GPIO; liefert aktiv-LOW-Logik für die Referenzfahrt
<code>coil.c</code>	Schaltet den Elektromagneten (PD0) ein und aus
<code>term.c</code>	Registry-Muster: Jeder Befehl registriert einen Handler; beim Empfang eines vollständigen UART-Frames wird der passende Handler aufgerufen

Globale Datenstrukturen

`current_config_t` (globals.h)

```
typedef struct {
    uint16_t Minimal_Pause;    // Pause-Ticks (nach Overflow-Auflösung)
    uint16_t Minimal_Pulse;    // Puls-Ticks (nach Overflow-Auflösung)
    bool      RampIsDisabled;    // true = Rampe aktiv (Ramp_Disabled=false)
} current_config_t;
```

Diese Struktur wird von `calcPulsePause()` befüllt und von den Motor-ISRs gelesen.

Ramp-Arrays

Für jeden Motor (M1/M2/M3) und jeden Rampen-Schritt `i` (0...RAMP_NSTEPS) existieren folgende Arrays:

Array	Bedeutung
<code>Ramp_Mx_Pulse_Ticks[i]</code>	Pulsbreite (Ticks) in Rampen-Stufe <code>i</code> (16-bit Rest nach Overflow)
<code>Ramp_Mx_Pulse_Ticks_OF[i]</code>	Overflow-Zähler für Pulsbreite (wie viele Male 65535 Ticks abgezählt werden)
<code>Ramp_Mx_Pause_Ticks[i]</code>	Pausen-Breite (Ticks) in Rampen-Stufe <code>i</code>
<code>Ramp_Mx_Pause_Ticks_OF[i]</code>	Overflow-Zähler für Pausen-Breite
<code>Ramp_Mx_End_Rem_Ticks[i]</code>	Verbleibende Ticks im aktuellen Zustand am Ende von Segment <code>i</code>
<code>Ramp_Mx_End_Rem_Ticks_OF[i]</code>	Overflow-Zähler für End-Rest
<code>Ramp_Mx_Rem_Pending[i]</code>	true = ISR muss zuerst End-Rest von Segment <code>i-1</code> abarbeiten

Array	Bedeutung
<code>Ramp_Mx_Start_State[i]</code>	Zustand (HIGH=true/LOW=false) am Beginn von Segment i

Die `_Curr`-Varianten (z.B. `Ramp_Mx_Pause_Ticks_OF_Curr[i]`) werden von den ISRs dekrementiert und durch die Reset-Werte neu geladen.

Kompilierzeit-Konfigurationsflags

Definiert in `globals.h` (Motortiming, Rampe) und `platform.h` (Hardware, UART):

Flag	Standard	Wirkung
<code>DEBUG_MODE</code>	0	Aktiviert Debug-Pin-Überwachung (<code>DEBUG_MODE_ISR1/2/3</code>); deaktiviert die Hauptschleife
<code>TEST_SEQUENCE</code>	0	Führt beim Start einen automatisierten Bewegungstest aus (erfordert <code>DEBUG_MODE=1</code>)
<code>RAMP_MODE_NSTEP</code>	1	Aktiviert N-Schritt-Rampenprofil (Anlauf-Rampe); CH6-ISR inkrementiert den Ramp-Schritt-Index
<code>RAMP_MODE_END</code>	1	Aktiviert die Brems-Rampe am Ende der Bewegung (Prescaler-Reduktion bei <code>RAMP_END_PS1/RAMP_END_PS2</code> verbleibenden Schritten)
<code>RAMP_MODE_PREMIUM</code>	0	Alternative: lineare Rampe, in der ISR berechnet (aktuell nicht aktiv)
<code>OVERFLOW_HANDLING</code>	1	Aktiviert 32-Bit-Overflow-Zähler für Pausen > 65535 Ticks (in <code>motor_config.h</code>)
<code>ISR_MONITOR</code>	0	Gibt ISR-Timing auf Reserve-Debug-Pins aus
<code>SIM_SENSORS</code>	0	Simuliert Sensorsignale, wenn keine Hardware angeschlossen ist
<code>COMMAND_BYTE</code>	0	Experimentelles Feature: optionales Befehls-Byte am Anfang des UART-Frames
<code>ENABLE_STEP</code>	1	Kompiliert STEP-Pin-Makros ein
<code>ENABLE_DIR</code>	1	Kompiliert DIR-Pin-Makros ein
<code>ENABLE_EN</code>	1	Kompiliert ENABLE-Pin-Makros ein
<code>ENABLE_ROT</code>	1	Kompiliert Rotationsmotor-Support ein

Startsequenz

```

main()
├── controlInit()
│   ├── clkGating()      GPIO-Ports A...E einschalten
│   ├── uart2Init()      UART2 für Raspberry Pi initialisieren
│   └── motorInit()       FTM0, GPIO-Pins, Emergency-Stop, Ramp-Arrays

```

	sensorInit()	Endschalter-Pins konfigurieren
	coilInit()	Spulen-Pin konfigurieren
	ftm0EnableIRQ()	FTM0-IRQ im NVIC aktivieren (Prio 12)

	controlRun()	[Hauptschleife]
	warte auf UART-Befehl → term.c dispatcht → Handler	

Hardware

Mikrocontroller

Eigenschaft	Wert
Gerät	NXP Kinetis MK22F51212
Kern	ARM Cortex-M4 mit FPU
Kerntakt	120 MHz (CORECLOCK)
Bustakt	60 MHz (BUSCLOCK) — Quelle für FTM0
Flash	256 KB
Zielboards	MC_CAR (100-polig, PINID=0x08), TinyK22 (64-polig, PINID=0x05)
Board-Erkennung	Zur Laufzeit via <code>SIM→SDID & SIM_SDID_PINID_MASK</code>



Das Zielsystem wird zur Compile-Zeit über `#define PLATFORM AUTO` ausgewählt. Im AUTO-Modus wird die Board-Erkennung zur Laufzeit durchgeführt.

Pin-Belegung

Schrittmotoren (STEP-Pins)

Die STEP-Pins werden als GPIO-Ausgänge konfiguriert (MUX=1). Die FTM0-Kanäle laufen im Output-Compare-Modus ohne Hardware-Pin-Ausgabe (ELsx=00) — das Toggle erfolgt manuell in der ISR über PSOR/PCOR.

Motor	STEP-Pin	FTM0-Kanal	Pull / DSE
Motor 1 (Achse 1)	PA4	CH1	Pull-down, DSE aktiv
Motor 2 (Achse 2)	PA5	CH2	Pull-down, DSE aktiv
Motor 3 (Achse 3)	PD4	CH4	Pull-down, DSE aktiv
Rotationsmotor	PD5	CH5	Pull-down, DSE aktiv

Richtungs-Pins (DIR)

Alle DIR-Pins liegen auf Port B. `FWD` = HIGH (PSOR), `REV` = LOW (PCOR). Nach dem Setzen der Richtung muss mindestens `DIR_MIN_DELAY_TIME_MS = 1 ms` gewartet werden.

Motor	DIR-Pin
Motor 1	PB0
Motor 2	PB1
Motor 3	PB2

Motor	DIR-Pin
Rotationsmotor	PB3

Enable-Pins (EN)

Alle EN-Pins liegen auf Port B. Der Treiber ist **aktiv LOW** — `ENABLE()` setzt PCOR, `DISABLE()` setzt PSOR. Nach dem Aktivieren muss mindestens `ENABLE_MIN_DELAY_TIME_MS = 400 ms` gewartet werden, bevor Schritte ausgegeben werden (Treibervorschrift).

Motor	EN-Pin
Motor 1	PB16
Motor 2	PB17
Motor 3	PB18
Rotationsmotor	PB19

Sensoren (Endschalter)

Die Endschalter sind **aktiv LOW** (Pull-up aktiv). Ein ausgelöster Sensor liefert logisch 0.

Sensor	Pin	Konfiguration
Sensor 1 (Achse 1)	PC8	Digitaleingang, Pull-up, aktiv LOW
Sensor 2 (Achse 2)	PC9	Digitaleingang, Pull-up, aktiv LOW
Sensor 3 (Achse 3)	PC10	Digitaleingang, Pull-up, aktiv LOW

Spule (Elektromagnet)

Signal	Pin
Spulensteuerung	PD0 (GPIO-Ausgang)

Debug- / Reserve-Pins

Diese Pins werden bei `DEBUG_MODE=1` oder `ISR_MONITOR=1` für Timing-Messungen per Oszilloskop genutzt.

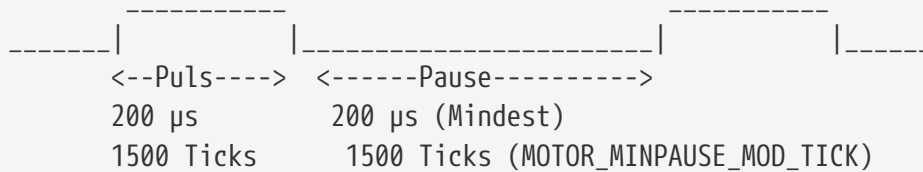
Signal	Pin	Typische Nutzung
RES0 / Debug 0	PA13	—
RES1 / Debug 1	PA12	ISR-Eingang/-Ausgang (CH1/CH2/CH6)
RES2 / Debug 2	PD1	Spulen-Monitoring (<code>RES1_GPIO_LOW</code> = Spule aus)
RES3 / Debug 3	PD6	—

Timer — FTM0

Timer-Takt und Auflösung

Parameter	Wert
Bustakt (Quelle)	60 MHz (<code>CLK_SRC_GLOBAL = 1</code>)
Prescaler	8 (<code>PS_GLOBAL = 3 → 2^3</code>)
Effektiver Timer-Takt	7,5 MHz
Tick-Dauer	133,33 ns
Timer-Modulo	0xFFFF = 65535 Ticks
Max. Periode ohne Overflow	$65535 \times 133,33 \text{ ns} \approx 8,74 \text{ ms}$

Schrittsignal-Timing



Parameter	Wert	Definition
Pulsbreite	200 µs / 1500 Ticks	<code>MOTOR_PULSE_US = 200,</code> <code>MOTOR_PULSE_MOD_TICK</code>
Mindest-Pause	200 µs / 1500 Ticks	<code>MIN_STEP_DISTANCE_US = 200,</code> <code>MOTOR_MINPAUSE_MOD_TICK</code>
Slow-Mode-Faktor	4×	<code>SLOW_MODE_FACTOR = 4</code> (Pick & Place Modus)

FTM0-Kanal-Zuordnung

Kanal	Funktion	Details
CH0	Intern (ungenutzt)	Wird im ISR-Dispatcher geprüft, hat keinen aktiven Handler
CH1	Schrittpuls Motor 1	PA4, <code>FTM0CH1_IRQHandler</code>
CH2	Schrittpuls Motor 2	PA5, <code>FTM0CH2_IRQHandler</code>
CH3	Ungenutzt	Default-Handler (bkpt bei unerwarteter Auslösung)
CH4	Schrittpuls Motor 3	PD4, <code>FTM0CH4_IRQHandler</code>
CH5	Schrittpuls Rotationsmotor	PD5, <code>FTM0CH5_IRQHandler</code>

Kanal	Funktion	Details
CH6	Ramp-Schritt-Inkrementierer	Nur aktiv bei <code>RAMP_MODE_NSTEP=1</code> ; löst beim Ablauf jedes Ramp-Segments aus und inkrementiert <code>Ramp_Step_Curr</code>
CH7	Emergency Stop	Immer aktiv; stoppt dauerhaft alle Motoren bei Auslösung

NVIC-Prioritäten

Interrupt	Priorität
FTM0 (alle Kanäle)	12 (<code>PRI0_FTM0</code>)
UART0/1/2	8 (<code>PRI0_UART0/1/2</code>)



Niedrigere Zahl = höhere Priorität. UART-Interrupts haben Priorität vor FTM0.

Kommunikation

UART-Schnittstellen

Das System nutzt zwei serielle Schnittstellen:

UART	Baudrate	Zweck
UART2	115200 Baud	Hauptkommunikation mit Raspberry Pi (Bewegungsbefehle)
UART0/1	57600 Baud	Debug-Terminal (<code>TERMINAL_BAUDRATE</code> in <code>platform.h</code>)

Kommunikationsprotokoll (UART2)

Die Kommunikation zwischen Raspberry Pi und Mikrocontroller läuft über UART2. Befehle werden als strukturiertes Binärpaket in Form der `ReceivedCommand`-Struktur übertragen:

```
typedef struct {  
    int32_t Steps1;      // Schrittzahl Achse 1 (negativ = Rückwärts)  
    int32_t Steps2;      // Schrittzahl Achse 2  
    int32_t Steps3;      // Schrittzahl Achse 3  
    int32_t StepsRot;     // Schrittzahl Rotationsmotor  
    bool    ActCoil;      // true = Elektromagnet einschalten  
    bool    ErrorHandling; // true = Fehlerbehandlungsmodus aktiv  
} ReceivedCommand;
```

Die Struktur wird komplett (alle Felder) pro Bewegungsbefehl übertragen. Der Mikrocontroller führt nach Empfang die folgenden Schritte aus:

1. Richtungspins für alle Achsen setzen (basierend auf Vorzeichen der Steps)
2. `calcPulsePause()` aufrufen → Berechnet Timer-Ticks und Ramp-Arrays
3. FTM0-Kanal-Interrupts für alle beteiligten Achsen aktivieren
4. Spule schalten (falls `ActCoil` gesetzt)
5. FTM0-Takt starten (`ftm0StartClk`)

Befehlssatz (Textkommunikation)

Über den Debug-UART (UART0/1) können Text-Befehle gesendet werden. `term.c` parst diese und ruft registrierte Handler auf:

Befehl	Beschreibung
<code>start</code>	Hauptbetriebsschleife starten (wartet auf Raspberry-Pi-Befehle)

Befehl	Beschreibung
<code>init</code>	Alle Achsen über Endschalter in die Ausgangsposition fahren (Referenzfahrt)
<code>camera</code>	Kameraaufnahme auslösen (Signal an Raspberry Pi)
<code>puzzleBegin</code>	Puzzle-Lösungssequenz starten
<code>puzzleEnd</code>	Puzzle-Lösungssequenz beenden

Terminal-Handler (`term.c`)

`term.c` implementiert einen Befehls-Dispatcher nach dem Registry-Muster:

- Beim Start registriert jedes Modul seine Handler-Funktion mit einem Schlüsselwort
- Der UART-Interrupt empfängt Zeichen und puffert sie bis zum Zeilenende (`\n` oder `\r`)
- Der Dispatcher vergleicht den empfangenen String mit allen registrierten Schlüsselwörtern
- Bei Treffer wird der zugehörige Handler aufgerufen

```
// Beispiel-Registrierung (aus control.c / motor.c):
termRegister("init", handleInit);
termRegister("start", handleStart);
```

Fehlerbehandlung

- Bei ungültigem Befehl gibt `term.c` den Fehlercode `EC_INVALID_CMD` zurück
- Der Emergency-Stop-Handler (CH7-ISR) sendet "ERROR" über `termWrite()` und hält das System dauerhaft an
- Die letzten ausgeführten Schritte/Richtungen werden in `M1_Last_Step`, `M2_Last_Step`, `M3_Last_Step`, `MR_Last_Step` (und `_Dir`-Varianten) gespeichert für die Fehler-Diagnose

Motorsteuerung

Dieses Kapitel beschreibt alle Aspekte der Schrittmotorsteuerung: FTM0-Konfiguration, ISR-Mechanismus, Überlaufbehandlung, Rundungsfehler-Korrektur, Anlauframpe, Bremsrampe, Referenzfahrt und Notabschaltung.

FTM0 Output-Compare — Grundprinzip

Kanalkonfiguration

Alle Motor-Kanäle (CH1/CH2/CH4/CH5/CH6) werden im **Software-Output-Compare-Modus** betrieben:

```
// motor.c – motorInit():
FTM0->CONTROLS[MOTOR1_STEP_TIMER_CHNL].CnSC =
    FTM_CnSC_MSB(0) | FTM_CnSC_MSA(1) |    // MSx=01: Output-Compare
    FTM_CnSC_ELSB(0) | FTM_CnSC_ELSA(0);    // ELSx=00: kein Hardware-Pin-Toggle
```

Bitfeld	Wert	Bedeutung
MSx (MSB:MSA)	01	Output-Compare-Modus (kein PWM)
ELsx (ELSB:ELSA)	00	Hardware-Toggle deaktiviert — kein FTM-Output am Pin
CHIE	0 → 1	Channel-Interrupt enable (wird kurz vor Bewegungsstart gesetzt)
CHF	—	Channel-Flag: wird gesetzt wenn $CNT == CnV$; muss in ISR manuell gelöscht werden

Die Pins PA4/PA5/PD4/PD5 bleiben als GPIO (MUX=1) konfiguriert. Das Toggle erfolgt ausschliesslich manuell in der ISR über **PSOR/PCOR**.

CnV-Inkrement-Prinzip

Der FTM0-Zähler läuft frei von 0 bis 0xFFFF und wrappet dann zurück auf 0. Das Interrupt-Timing wird durch **relative CnV-Inkremente** gesteuert:

```
// In der ISR: nächsten Interrupt in "pause" Ticks planen:
FTM0->CONTROLS[x].CnV += pause;    // CnV = aktuelles_CnV + pause (modulo 65536)
```

Dieses Addieren (nicht absolutes Setzen) funktioniert korrekt über Wrap-Arounds. Wenn z.B. $CnV = 60000$ und $pause = 10000$, ergibt sich $CnV = 70000 \bmod 65536 = 4464$. Der Interrupt feuert dann bei $CNT == 4464$ im nächsten Zähler-Umlauf.

Timer-Parameter

Parameter	Wert
Bustakt	60 MHz (<code>CLK_SRC_GLOBAL = 1</code>)
Prescaler	8 (<code>PS_GLOBAL = 3</code> → 2^3)
Effektiver Takt	7,5 MHz
Tick-Periode	133,33 ns
Modulo	0xFFFF = 65535 (16-Bit Freerun)
Max. Pause ohne Overflow	$65535 \times 133,33 \text{ ns} = 8,74 \text{ ms}$
Pulsbreite	1500 Ticks = 200 µs (<code>MOTOR_PULSE_MOD_TICK</code>)
Mindest-Pause	1500 Ticks = 200 µs (<code>MOTOR_MINPAUSE_MOD_TICK</code>)

ISR-Dispatcher

FTM0 hat genau einen Hardware-Interrupt-Vektor (`FTM0_IRQHandler`). Der Dispatcher in `ftm0.c` prüft für jeden Kanal, ob sowohl das Interrupt-Flag (CHF) als auch das Interrupt-Enable (CHIE) gesetzt sind, und ruft die zugehörige Sub-ISR auf:

```
#define CHF_CHIE_MASK (FTM_CnSC_CHF_MASK | FTM_CnSC_CHIE_MASK)
#define TOF_TOIE_MASK (FTM_SC_TOF_MASK | FTM_SC_TOIE_MASK)

void FTM0_IRQHandler(void) {
    if ((FTM0->CONTROLS[0].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH0_IRQHandler();
    if ((FTM0->CONTROLS[1].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH1_IRQHandler();
    if ((FTM0->CONTROLS[2].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH2_IRQHandler();
    if ((FTM0->CONTROLS[3].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH3_IRQHandler();
    if ((FTM0->CONTROLS[4].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH4_IRQHandler();
    if ((FTM0->CONTROLS[5].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH5_IRQHandler();
    if ((FTM0->CONTROLS[6].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH6_IRQHandler();
    if ((FTM0->CONTROLS[7].CnSC & CHF_CHIE_MASK) == CHF_CHIE_MASK)
        FTM0CH7_IRQHandler();
    if ((FTM0->SC & TOF_TOIE_MASK) == TOF_TOIE_MASK)
        FTM0TOF_IRQHandler();
}
```



Die Bedingung **CHF AND CHIE** ist zwingend. Würde nur CHF geprüft, könnten Interrupts verpasster Kanäle fälschlicherweise ausgelöst werden. Würde nur CHIE

geprüft, würde ein Kanal dessen Interrupt noch nicht gefeuert hat den Handler aufrufen.

Jede per-Kanal ISR muss als **erste Anweisung** das CHF-Flag löschen:

```
void FTM0CH2_IRQHandler(void) {  
    FTM0->CONTROLS[2].CnSC &= ~FTM_CnSC_CHF(1); // CHF löschen – NICHT verschieben!  
    // ...  
}
```

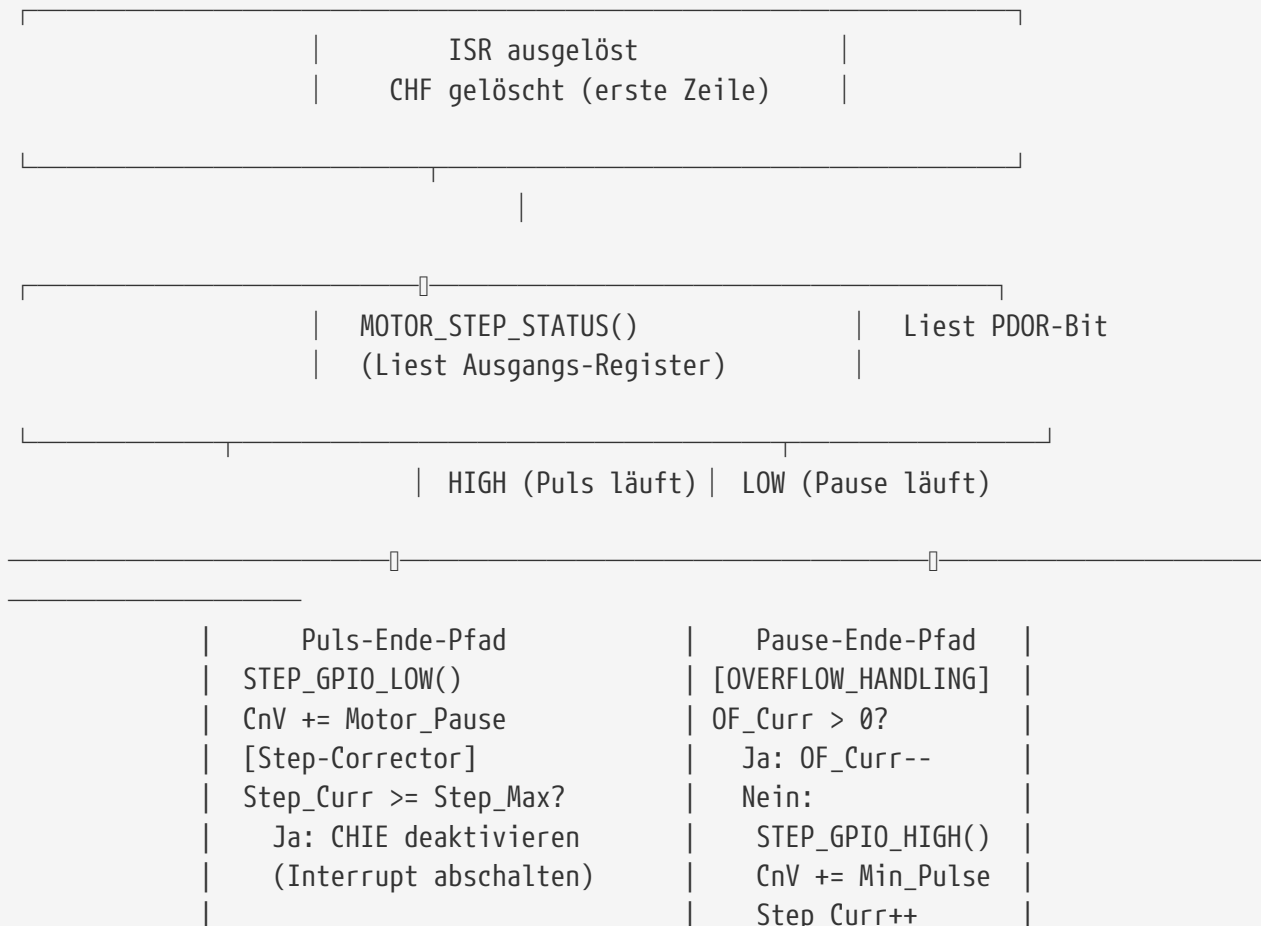


Das CHF-Flag darf **nicht** erst am Ende der ISR gelöscht werden. Wenn der nächste Interrupt-Zeitpunkt bereits vor dem Ende der ISR liegt (sehr schnelle Motoren), würde CHF sofort wieder gesetzt — ohne das vorherige Löschen wäre die Auslösereihenfolge korrumpiert.

Schrittpuls-Erzeugung (Normalbetrieb)

Zwei-Zustands-Automat (CH1/CH2/CH4)

Jeder Motor-Kanal implementiert einen einfachen Zustandsautomaten basierend auf dem aktuellen GPIO-Ausgangszustand (PDOR-Register):



Konkretes Timing-Diagramm

FTM0 CNT: 0 1500 3000 4500 6000 7500 9000 10500

GPIO PA5: |□□□□| |□□□□| |□□□□| |□□□□|...

 Puls1 Pause1 Puls2 Pause2 Puls3

CnV-Ablauf:

```
Start:  CnV = FIRST_PULSE_START_MOD (= 1)
ISR1:   GPIO_HIGH, CnV += 1500 → CnV = 1501 (Puls)
ISR2:   GPIO_LOW,  CnV += 1500 → CnV = 3001 (Pause)
ISR3:   GPIO_HIGH, CnV += 1500 → CnV = 4501
ISR4:   GPIO_LOW,  CnV += 1500 → CnV = 6001
```

Rotationsmotor (CH5)

Der Rotationsmotor `FTM0CH5_IRQHandler` ist identisch aufgebaut, jedoch ohne Ramp-Unterstützung und mit halber Mindest-Pause:

```
// motor.c – moveWay():
MotorRot_Pause = MOTOR_MINPAUSE_MOD_TICK / 2; // 750 Ticks = 100 µs
```

Der Rotationsmotor läuft damit doppelt so schnell wie die Linearachsen im Normalbetrieb.

Overflow-Behandlung (OVERFLOW_HANDLING)

Problem

FTM0 ist 16-Bit. Der maximale CnV-Inkrement pro ISR-Aufruf ist 65535 Ticks = 8,74 ms. Für langsame Achsen (viele Schritte → kleine Pause nötig, aber Motor mit wenigsten Schritten braucht grosse Pause) kann die berechnete Pause > 65535 Ticks sein.

Auflösung in `calcPulsePause()`

Die Pause wird in vollständige 65535-Tick-Blöcke und einen Rest zerlegt:

```
// Beispiel: tmp_M2_Pause = 130000 Ticks
do {
    if (tmp_M2_Pause >= UINT16_MAX) {           // 130000 >= 65535
        tmp_M2_Pause -= UINT16_MAX;             // 130000 - 65535 = 64465
        tmp_M2_OF_CNT++;                       // OF_CNT = 1
    }
}
```

```

} while (tmp_M2_Pause >= UINT16_MAX);
// Ergebnis: Motor2_Step_OF = 1, Motor2_Pause = 64465

```

Abarbeitung in der ISR

```

// Pause-Ende-Pfad in FTM0CH2_IRQHandler:
if (Motor2_Step_OF_Curr > 0) {
    // CnV NICHT verändern – nächste Auslösung = CNT + 65535 Ticks (voller Wrap)
    Motor2_Step_OF_Curr--;
} else {
    // Overflow vollständig abgewartet: Puls ausgeben
    MOTOR2_STEP_GPIO_HIGH();
    FTM0->CONTROLS[2].CnV += Current_Config.Minimal_Pulse;
    Motor2_Step_Curr++;
    Motor2_Step_OF_Curr = Motor2_Step_OF;    // Overflow-Zähler für nächsten Schritt
    laden
}

```

Initiale Pause (halbe Pause)

Damit alle Motoren zeitlich synchron starten, wird der erste CnV-Wert auf die **halbe** Pause gesetzt. Der Master-Motor (meiste Schritte) startet bei **FIRST_PULSE_START_MOD = 1** (sofort), alle anderen bei **FIRST_PULSE_START_MOD + Pause/2**. Dadurch trifft der erste Puls des langsamsten Motors in der Mitte der ersten Pulsperiode des schnellsten Motors ein:

```

// Berechnung der initialen CnV:
if (tmp_M2_OF_CNT % 2 == 0) {
    tmp_M2_Pause_init_CNV = tmp_M2_Pause / 2;
} else {
    // Bei ungeradem OF: die halbe Pause überlappt einen Overflow
    tmp_M2_Pause_init_CNV = (UINT16_MAX / 2) + (tmp_M2_Pause / 2);
}

```

Analog wird **Motor_Step_OF_Curr** auf **OF_CNT / 2** initialisiert (halber Overflow-Zähler).

Step-Corrector (Rundungsfehler-Kompensation)

Ursache des Problems

Die Berechnung der Pause für nicht-Master-Motoren erfolgt durch ganzzahlige Division:

```

totalTicks = (min_pause + min_pulse) * Mot_Master;    // z.B. 3000 x 1000 = 3'000'000
totalTicksPauseM2 = totalTicks - (min_pulse * Mot2); // 3'000'000 - 1500x700 =
1'950'000
tmp_M2_Pause = totalTicksPauseM2 / Mot2;              // 1'950'000 / 700 = 2785 (Rest:

```

Der Rest von 500 Ticks würde sich über 700 Schritte als Zeitversatz akkumulieren: Am Ende wäre Motor 2 um 500 Ticks früher fertig als Motor 1. Bei 700 Schritten entspricht das einem Positions-Offset.

Algorithmus

`calcPulsePause()` verteilt den Restfehler gleichmässig über alle Schritte mittels eines Modulo-basierten Korrektors:

```
// errorStepperM2 = 500 Ticks Restfehler bei 700 Schritten
errorStepperM2 = totalTicks - ((tmp_M2_Pause + min_pulse) * Mot2);
// = 3'000'000 - (2785 + 1500) × 700 = 500

if (errorStepperM2 > 0) {
    // Erste Korrekturstufe: alle (700/500 + 1) = 2 Schritte → +1 Tick
    Motor2_Step_Corrector[0] = (Mot2 / errorStepperM2) + 1;    // = 2

    for (int i = 0; i < NUM_CORRECTOR_LOOPS - 1; i++) {
        if (errorStepperM2 > 0 && Motor2_Step_Corrector[i] > 0) {
            // Verbleibender Fehler nach Anwenden von Corrector[i]:
            errorStepperM2 -= (1.0 / Motor2_Step_Corrector[i]) * Mot2;
            Motor2_Step_Corrector[i+1] = Mot2 / errorStepperM2;
        }
    }
}
```

Anwendung in der ISR (am Ende des Puls-Ende-Pfads)

```
// Nach: CnV += Motor2_Pause
for (int i = 0; i < NUM_CORRECTOR_LOOPS; i++) {
    if (Motor2_Step_Corrector[i]) {
        if ((Motor2_Step_Curr % Motor2_Step_Corrector[i]) == 0) {
            FTM0->CONTROLS[2].CnV += 1;    // Eintrag aktiv?
                                           // Vielfaches?
                                           // +1 Tick
        }
    }
}
```

Wenn `Motor2_Step_Corrector[0] = 2`: bei jedem 2. Schritt (Step 2, 4, 6, ...) wird 1 extra Tick addiert. Dies summiert sich auf $700/2 = 350$ Ticks über die gesamte Bewegung — der Fehler von 500 Ticks wird so zu 350 korrigiert. Die weiteren Korrektur-Stufen (Corrector[1], [2], ...) verteilen den verbleibenden Restfehler von 150 Ticks weiter.

`NUM_CORRECTOR_LOOPS = 10` erlaubt bis zu 10 kaskadierte Korrekturen für beliebig geringe Endresiduen.



Der Corrector greift in der ISR nur im **Puls-Ende-Pfad** (GPIO_LOW), also einmal pro vollständigem Schritt — nicht in der Puls-Start-Phase.

moveWay() — Vollständiger Ablauf

Die Funktion `moveWay()` in `motor.c` ist die zentrale Bewegungsroutine und wird von `control.c` aufgerufen.

```
moveWay(mot1, mot2, mot3, RotSteps, Ramp_Disabled)
|
|— 1. Absolutwerte und Richtung berechnen
|   mot_Abs = |mot|, Dir = FWD/REV
|   Motor_Step_Max = mot_Abs    (globale Max-Schrittzahl für ISR)
|
|— 2. Enable-Pins aktivieren (MOTOR_EN_ENABLE → PB16/17/18/19 LOW)
|   waitMs(400)                ← ENABLE_MIN_DELAY_TIME_MS
|
|— 3. Richtungs-Pins setzen (MOTOR_DIR_FWD/REV → PB0/1/2/3)
|   waitMs(1)                  ← DIR_MIN_DELAY_TIME_MS
|
|— 4. calcPulsePause()
|   → Berechnet Motor_Pause, Motor_Step_OF für alle 3 Achsen
|   → Füllt Motor_Step_Corrector[]
|   → [wenn Ramp_Disabled] Berechnet alle Ramp-Arrays (Pulse/Pause/EndRem Ticks)
|   → Setzt initiale CnV-Werte
|
|— 5. ftm0StopClk()              ← Takt anhalten (sicher für CnV-Schreiben)
|
|— 6. CnV und CHF für alle Kanäle setzen
|   [Ramp_Disabled=true]:  CnV = Ramp_Mx_End_Rem_Ticks[0], CH6.CnV =
Ramp_Step_Ticks[1]
|   [Ramp_Disabled=false]: CnV = FIRST_PULSE_START_MOD (= 1), Ramp_Step_Curr =
NSTEPS+1
|   FTM0->CNT = 0              ← Counter resettten
|
|— 7. CHF-Flags löschen, CHIE für aktive Motoren setzen
|   [Ramp_Disabled=true]:  auch CH6.CHIE setzen
|
|— 8. Rotationsmotor konfigurieren (falls RotSteps != 0)
|   MotorRot_Pause = MOTOR_MINPAUSE_MOD_TICK / 2 (750 Ticks)
|   ftm0StopClk(), CNT=0, CnV = FIRST_PULSE_START_MOD
|
|— 9. ftm0EnableIRQ()           ← NVIC aktivieren (Prio 12)
|   ftm0StartClk(CLK_SRC, PS_GLOBAL) ← Takt starten → ISRs laufen ab hier
|
|— 10. Warte-Schleife (blockierend, in main context)
|   while (true) {
|       [Ramp_Disabled=true] End-Rampe prüfen und ftm0ReducePS() aufrufen
|       Alle Step_Curr >= Step_Max AND alle CHIE == 0? → break
```

```

    }

    11. initGlobalsMove() + initGlobalsRot()
        Alle Ramp-Arrays und Overflow-Zähler zurücksetzen

    12. waitMs(20)                ← Letzten Puls sicher beenden (100 x 200µs/1000)

    13. resetMoveTimers() + resetRotTimers()
        ftm0StopIRQ(), alle Step_Curr/Max = 0

```

Ramp_Disabled-Parameter — Semantik

Der Name ist irreführend. Die Bedeutung ist:

Ramp_Disabled	Verhalten
true	Rampe aktiv: Normale Betriebsgeschwindigkeit, NSTEP-Anlauframpe wird berechnet und ausgeführt (CH6 aktiv), RAMP_END-Bremsrampe aktiv
false	Langsam-Modus: Geschwindigkeit um SLOW_MODE_FACTOR = 4 reduziert, keine Anlauframpe (Ramp_Step_Curr = NSTEPS+1, direkt in Constant-Pfad), kein CH6

Der Langsam-Modus wird für Pick-&-Place-Operationen verwendet, bei denen Präzision wichtiger als Geschwindigkeit ist.

N-Schritt-Rampenprofil (RAMP_MODE_NSTEP)

Konzept: Zeitsegmente mit sinkender Dauer

Die Rampe teilt die Anlaufphase in **RAMP_NSTEPS = 10** Segmente. Jedes Segment hat eine feste Dauer in Ticks, innerhalb derer der Motor mit der zugehörigen (skalierten) Puls/Pause-Breite läuft. Das erste Segment ist am längsten (langsamste Geschwindigkeit), das letzte am kürzesten (Zielgeschwindigkeit):

$$\text{Segment-Faktor}[i] = 1 + (\text{RAMP_NSTEPS} - i + 1) \times (\text{RAMP_NSTEPS_STEP_PERC} / 100.0)$$

RAMP_NSTEPS_STEP_PERC = 100 → jede Stufe ist 1.0 x RAMP_NSTEPS_STEPS länger

Stufe i	Faktor	Dauer (Ticks)	Dauer (ms)
10	11.0	220'000	29,3 ms
9	10.0	200'000	26,7 ms
8	9.0	180'000	24,0 ms
7	8.0	160'000	21,3 ms
6	7.0	140'000	18,7 ms
5	6.0	120'000	16,0 ms
4	5.0	100'000	13,3 ms


```

    } else {
        Ramp_M2_Rem_Pending[Ramp_Step_Curr] = false;
        // Wechsel in Start_State des neuen Segments
    }
}

```

CH6-ISR: Ramp-Schritt-Inkrementierer

CH6 feuert genau am Ende jedes Ramp-Segments und koordiniert den atomaren Übergang:

```

void FTM0CH6_IRQHandler(void) {
    FTM0->CONTROLS[6].CnSC &= ~FTM_CnSC_CHF(1);

    if (Ramp_Step_Curr < RAMP_NSTEPS) {
        if (Ramp_Step_Ticks_OF_Curr[Ramp_Step_Curr] > 0) {
            Ramp_Step_Ticks_OF_Curr[Ramp_Step_Curr]--; // Segment-Overflow noch aktiv
        } else {
            // === Atomarer Segment-Übergang ===
            FTM0->SC &= ~(CLKS | PS); // Takt STOPPEN (atomar)
            FTM0->CONTROLS[6].CnV = Ramp_Step_Ticks[Ramp_Step_Curr];

            Ramp_Step_Curr++; // Nächste Stufe aktivieren

            // Motor-CnVs auf End-Rem-Ticks des neuen Segments setzen:
            FTM0->CONTROLS[CH1].CnV = Ramp_M1_End_Rem_Ticks[Ramp_Step_Curr-1];
            FTM0->CONTROLS[CH2].CnV = Ramp_M2_End_Rem_Ticks[Ramp_Step_Curr-1];
            FTM0->CONTROLS[CH4].CnV = Ramp_M3_End_Rem_Ticks[Ramp_Step_Curr-1];

            // Overflow-Zähler für neues Segment laden:
            Ramp_M1_End_Rem_Ticks_OF_Curr[Ramp_Step_Curr-1] =
Ramp_M1_End_Rem_Ticks_OF[...];
            // [analog M2, M3]

            FTM0->CNT = 0;
            FTM0->SC = CLKS(CLK_SRC_GLOBAL) | PS(PS_GLOBAL); // Takt NEUSTARTEN
        }
    } else {
        // === Letzte Stufe abgeschlossen ===
        FTM0->SC &= ~(CLKS | PS);
        Ramp_Step_Curr++; // Ramp_Step_Curr > RAMP_NSTEPS →
Normalbetrieb
        FTM0->CONTROLS[6].CnSC &= ~CHIE; // CH6-Interrupt permanent deaktivieren

        // Motoren auf ihre End-Rem-Ticks[RAMP_NSTEPS] setzen:
        FTM0->CONTROLS[CH1].CnV = Ramp_M1_End_Rem_Ticks[RAMP_NSTEPS];
        FTM0->CONTROLS[CH2].CnV = Ramp_M2_End_Rem_Ticks[RAMP_NSTEPS];
        FTM0->CONTROLS[CH4].CnV = Ramp_M3_End_Rem_Ticks[RAMP_NSTEPS];

        FTM0->CNT = 0;
        FTM0->SC = CLKS(CLK_SRC_GLOBAL) | PS(PS_GLOBAL);
    }
}

```

```
}
}
```



Das Stoppen des FTM0-Takts während des Segment-Übergangs ist zwingend. Würde der Zähler während dem Schreiben der CnV-Werte weiterlaufen, könnten CNT bereits an einzelnen CnV-Werten vorbeigehen, bevor alle geschrieben sind — das würde zu verpassten Interrupts führen.

Motor-ISR im Ramp-Modus (vereinfacht für CH2)

```
void FTM0CH2_IRQHandler(void) {
    FTM0->CONTROLS[2].CnSC &= ~FTM_CnSC_CHF(1);

    static bool ramp_motor_state; // true = GPIO aktuell HIGH (Puls)

    if (Ramp_Step_Curr <= RAMP_NSTEPS) {
        // === Ramp-Modus ===
        if (Ramp_M2_Rem_Pending[Ramp_Step_Curr]) {
            // Reste vom vorherigen Segment abarbeiten
            if (Ramp_M2_End_Rem_Ticks_OF_Curr[Ramp_Step_Curr-1] > 0) {
                Ramp_M2_End_Rem_Ticks_OF_Curr[Ramp_Step_Curr-1]--;
            } else {
                Ramp_M2_Rem_Pending[Ramp_Step_Curr] = false;
                if (Ramp_M2_Start_State[Ramp_Step_Curr] == false) {
                    MOTOR2_STEP_GPIO_HIGH(); // Neues Segment beginnt mit Puls
                    FTM0->CONTROLS[2].CnV += Ramp_M2_Pulse_Ticks[Ramp_Step_Curr];
                    ramp_motor_state = true;
                    Motor2_Step_Curr++;
                } else {
                    MOTOR2_STEP_GPIO_LOW(); // Neues Segment beginnt mit Pause
                    FTM0->CONTROLS[2].CnV += Ramp_M2_Pause_Ticks[Ramp_Step_Curr];
                    ramp_motor_state = false;
                }
            }
        }
        else if (ramp_motor_state) {
            // GPIO HIGH → Puls läuft
            if (Ramp_M2_Pulse_Ticks_OF_Curr[Ramp_Step_Curr] > 0) {
                Ramp_M2_Pulse_Ticks_OF_Curr[Ramp_Step_Curr]--; // Overflow abwarten
            } else {
                MOTOR2_STEP_GPIO_LOW();
                FTM0->CONTROLS[2].CnV += Ramp_M2_Pause_Ticks[Ramp_Step_Curr];
                ramp_motor_state = false;
                Ramp_M2_Pulse_Ticks_OF_Curr[Ramp_Step_Curr] =
Ramp_M2_Pulse_Ticks_OF[...];
            }
        }
        else {
            // GPIO LOW → Pause läuft
            if (Ramp_M2_Pause_Ticks_OF_Curr[Ramp_Step_Curr] > 0) {
                Ramp_M2_Pause_Ticks_OF_Curr[Ramp_Step_Curr]--;
            }
        }
    }
}
```

```

    } else {
        MOTOR2_STEP_GPIO_HIGH();
        FTM0->CONTROLS[2].CnV += Ramp_M2_Pulse_Ticks[Ramp_Step_Curr];
        Motor2_Step_Curr++;
        ramp_motor_state = true;
        Ramp_M2_Pause_Ticks_OF_Curr[Ramp_Step_Curr] =
Ramp_M2_Pause_Ticks_OF[...];
    }
}
} else {
    // === Normalbetrieb (nach Ramp-Ende) ===
    // Drain End_Rem_Ticks des letzten Segments:
    if (Ramp_M2_End_Rem_Ticks_OF_Curr[RAMP_NSTEPS] > 0) {
        Ramp_M2_End_Rem_Ticks_OF_Curr[RAMP_NSTEPS]--;
    } else {
        // → Standard Zwei-Zustands-Automat (siehe oben)
    }
}
}
}

```

End-Rampe (**RAMP_MODE_END**)

Konzept

Am Ende der Bewegung, wenn die Restschrittzahl des Master-Motors bestimmte Schwellwerte unterschreitet, wird der FTM0-Prescaler erhöht. Ein höherer Prescaler bedeutet einen langsameren Timertakt — gleiche CnV-Inkremente erzeugen längere Pausen — der Motor bremst.

Schwellwert	rem_steps	Prescaler	Effektiver Takt
Normal	> 150	PS_GLOBAL = 3	7,5 MHz
Stufe 1	51...150	PS_GLOBAL+1 = 4	3,75 MHz (2× langsamer)
Stufe 2	1...50	PS_GLOBAL+2 = 5	1,875 MHz (4× langsamer)
Stufen 3/4	0	(deaktiviert, PS3=0)	—

Die Prüfung erfolgt in der Warte-Schleife in `moveWay()` (im main context, nicht in der ISR):

```

// moveWay() – Warte-Schleife:
bool reduce[4] = {true, true, true, true}; // jede Stufe nur einmal schalten

while (true) {
    if (Ramp_Disabled) { // End-Rampe nur im Rampen-Modus
        if (mostMotor == 1) rem_steps = mot1_Abs - Motor1_Step_Curr;
        // [analog motoren 2/3]

        if ((RAMP_END_PS2 < rem_steps) && (rem_steps <= RAMP_END_PS1) && reduce[0]) {
            ftm0ReducePS(CLK_SRC_GLOBAL, PS_GLOBAL + 1);
            reduce[0] = false;
        }
    }
}

```

```

    } else if ((rem_steps <= RAMP_END_PS2) && reduce[1]) {
        ftm0ReducePS(CLK_SRC_GLOBAL, PS_GLOBAL + 2);
        reduce[1] = false;
    }
}
// Abbruchbedingung: alle Schritte fertig UND alle CHIE == 0
if ((Motor1_Step_Curr >= mot1_Abs) && ... &&
    ((CH1.CnSC & CHIE) == 0) && ...) {
    break;
}
}

```

ftm0ReducePS() — Atomarer Prescaler-Wechsel

Das Stoppen und Neustarten des Timers ist nötig, weil FTM0 den Prescaler nicht im laufenden Betrieb sicher wechseln kann. Die CnV-Anpassung stellt sicher, dass kein Interrupt verpasst wird:

```

void ftm0ReducePS(int CLK_Source, int Prescaler) {
    uint16_t val = FTM0->CNT;
    FTM0->SC &= ~(CLKS | PS); // Timer STOPPEN (CNT einfrieren)

    // CnV aller Motor-Kanäle relativ zu CNT=0 anpassen:
    uint16_t cnv = FTM0->CONTROLS[CH1].CnV;
    FTM0->CONTROLS[CH1].CnV = (cnv > val) ? (cnv - val) : 1u;
    // Wenn cnv <= val: der Compare hat bereits gefeuert aber ISR ist noch pending.
    // → CnV=1 sorgt dafür, dass die ISR sofort nach Neustart feuert statt 65535 Ticks
    // zu warten.
    FTM0->CONTROLS[CH1].CnSC &= ~CHF; // Flag löschen

    for (int i = 0; i < 10; i++) { __NOP(); } // NOP-Lücke – NICHT ENTFERNEN

    cnv = FTM0->CONTROLS[CH2].CnV;
    FTM0->CONTROLS[CH2].CnV = (cnv > val) ? (cnv - val) : 1u;
    FTM0->CONTROLS[CH2].CnSC &= ~CHF;

    for (int i = 0; i < 10; i++) { __NOP(); } // NOP-Lücke – NICHT ENTFERNEN

    cnv = FTM0->CONTROLS[CH4].CnV;
    FTM0->CONTROLS[CH4].CnV = (cnv > val) ? (cnv - val) : 1u;
    FTM0->CONTROLS[CH4].CnSC &= ~CHF;

    for (int i = 0; i < 20; i++) { __NOP(); } // 20 NOPs für CH4 – NICHT ENTFERNEN

    FTM0->CNT = 0;
    FTM0->SC = CLKS(CLK_Source) | PS(Prescaler); // Neustart mit neuem Prescaler
}

```



Die NOP-Schleifen zwischen den CnV-Korrekturen sind **zwingend** erforderlich. Sie

geben der Hardware ausreichend Zeit, um ausstehende Interrupt-Flags korrekt zu verarbeiten. Ohne diese NOPs werden in der End-Rampe Interrupts übersprungen, was zu falschem Motorverhalten (zu früher Stopp oder Verzögerungen) führt. Dies wurde empirisch ermittelt und dokumentiert.

Referenzfahrt (**moveToInitPos**)

Die Referenzfahrt wird bei jedem Start aufgerufen und bringt alle drei Linearachsen in die definierte Ausgangsposition (Nullpunkt).

Dreiphasiger Ablauf

Phase 1: Schnell zum Sensor (100 µs Toggle)

```
| Alle Motoren: DIR = REV |
| Schleife: waitUs(100) → TOGGLE alle aktiven |
| Sensor 1 (PC8, aktiv LOW) auslöst → M1 stop |
| Sensor 2 (PC9) auslöst → M2 stop |
| Sensor 3 (PC10) auslöst → M3 stop |
```

□

Phase 2: Backoff (weg vom Sensor)

```
| Alle Motoren: DIR = FWD |
| 800 Schritte mit waitUs(1000) Toggle |
| (= toggle_US x 10 = 100 x 10 = 1000 µs) |
```

□

Phase 3: Langsam zurück zum Sensor (1000 µs Toggle)

```
| Alle Motoren: DIR = REV |
| Schleife: waitUs(1000) → TOGGLE aktive |
| Sensor 1 auslöst → M1 stop (präziser Punkt) |
| Sensor 2 auslöst → M2 stop |
| Sensor 3 auslöst → M3 stop |
```

□

```
Motor_Step_Curr = 0 (alle Positionen = Nullpunkt)
```

Phase 3 läuft 10× langsamer als Phase 1, wodurch der Endschalter-Auslösepunkt mit höherer Präzision erfasst wird. Die 800 Backoff-Schritte stellen sicher, dass der Motor mit Sicherheitsabstand vom Sensor wegfährt, bevor er langsam zurückkommt.

Die Funktion wird mit `toggle_US = 100` aufgerufen (aus `motorInit()`):

```
moveToInitPos(100); // Phase 1: 100 µs/Toggle, Phase 2/3: 1000 µs/Toggle
```

Sensoren

Die Endschalter (PC8/PC9/PC10) sind **aktiv LOW** mit Pull-up konfiguriert. Ein ausgelöster Sensor liefert `SENSOR_STATUS() == 0`:

```
if (!SENSOR1_STATUS()) { m1_phase1 = false; } // Sensor ausgelöst → M1 stoppen
```

Synchronisation der drei Achsen

Master-Slave-Prinzip

`calcPulsePause()` bestimmt den Motor mit den meisten Schritten (Master). Dieser Motor bekommt die kürzestmögliche Pause (`MOTOR_MINPAUSE_MOD_TICK = 1500 Ticks = 200 µs`) und läuft damit mit Maximalgeschwindigkeit. Die Gesamtbewegungsdauer in Ticks ist:

```
TotalTicks = (min_pause + min_pulse) x Steps_Master  
             = (1500 + 1500) x Steps_Master  
             = 3000 x Steps_Master
```

Alle anderen Motoren erhalten eine längere Pause, so dass ihre Gesamtdauer ebenfalls `TotalTicks` beträgt:

```
Pause_M2 = (TotalTicks - min_pulse x Steps_M2) / Steps_M2  
          = (3000 x Steps_Master - 1500 x Steps_M2) / Steps_M2
```

Zahlenbeispiel

Parameter	Motor 1	Motor 2	Motor 3
Schritte	1000	700	400
Rolle	Master	Slave	Slave
Pause (Ticks)	1500	2786	6000

Parameter	Motor 1	Motor 2	Motor 3
Puls (Ticks)	1500	1500	1500
Gesamt/Schritt	3000	4286	7500
Gesamtdauer (Ticks)	3'000'000	3'000'200	3'000'000
Gesamtdauer (ms)	400	400	400

Motor 2 hat einen Rundungsfehler von 200 Ticks → wird durch Step-Corrector kompensiert.

Emergency Stop (FTM0 CH7)

Konfiguration

CH7 wird in `emergencyStop_Init()` als Output-Compare ohne Hardware-Pin konfiguriert. Der Auslösezeitpunkt muss im Setup gesetzt werden. CH7 ist das einzige CHIE, das von `ftm0StopIRQ()` **nicht** deaktiviert wird:

```
// ftm0.c – ftm0StopIRQ():
FTM0->CONTROLS[0].CnSC &= ~CHIE;
FTM0->CONTROLS[1].CnSC &= ~CHIE;
// ...
FTM0->CONTROLS[6].CnSC &= ~CHIE;
// Kanal 7 wird ABSICHTLICH nicht deaktiviert:
// FTM0->CONTROLS[7].CnSC &= ~CHIE; ← auskommentiert
```

CH7-ISR — Ablauf bei Auslösung

```
void FTM0CH7_IRQHandler(void) {
    FTM0->CONTROLS[7].CnSC &= ~CHF;

    termWrite("ERROR");           // Fehlermeldung über UART senden

    // Alle Motor-Pins sofort auf LOW:
    MOTOR1_STEP_GPIO_LOW();
    MOTOR2_STEP_GPIO_LOW();
    MOTOR3_STEP_GPIO_LOW();
    MOTORROT_STEP_GPIO_LOW();
    RES1_GPIO_LOW();              // Spule deaktivieren

    while (true) {
        // Endlosschleife: alle Kanal-Interrupts in jeder Iteration deaktivieren
        FTM0->CONTROLS[CH1].CnSC &= ~CHIE;
        FTM0->CONTROLS[CH2].CnSC &= ~CHIE;
        FTM0->CONTROLS[CH4].CnSC &= ~CHIE;
        FTM0->CONTROLS[CH5].CnSC &= ~CHIE;
        FTM0->CONTROLS[6].CnSC &= ~CHIE;    // Ramp-Inkrementierer
    }
}
```



```
// System ist dauerhaft blockiert – Hardware-Reset erforderlich
}
```



Das wiederholte Deaktivieren der CHIE-Bits in der Endlosschleife stellt sicher, dass auch falls ein CHIE durch eine Race-Condition zwischenzeitlich gesetzt wurde, es sofort wieder deaktiviert wird. Das System ist nach Emergency-Stop nicht mehr betriebsfähig und erfordert einen vollständigen Hardware-Reset (Power-Cycle oder NVIC-Reset via J-Link).

Globale Variablen — Lebenszyklus

Variable	Typ	Lebenszyklus
Motor_Step_Curr	int32_t	Wird in ISR inkrementiert; Reset durch <code>resetMoveTimers()</code>
Motor_Step_Max	int32_t	Gesetzt durch <code>moveWay()</code> aus <code>mot_Abs</code> ; Reset durch <code>resetMoveTimers()</code>
Motor_Step_OF	int32_t	Gesetzt durch <code>calcPulsePause()</code> ; Reset durch <code>initGlobalsMove()</code>
Motor_Step_OF_Curr	int32_t	Gesetzt auf OF/2 in <code>calcPulsePause()</code> ; in ISR dekrementiert; Reset durch <code>initGlobalsMove()</code>
Motor_Pause	uint16_t	Gesetzt durch <code>calcPulsePause()</code> (Rest nach Overflow); in ISR gelesen
Motor_Step_Corrector[]	int32_t[]	Gesetzt durch <code>calcPulsePause()</code> ; in ISR gelesen; Reset auf 0 durch <code>initGlobalsMove()</code>
Ramp_Step_Curr	uint16_t	Zählt 0...NSTEPS+1; gesetzt in <code>calcPulsePause()</code> oder <code>moveWay()</code> ; Reset durch <code>initGlobalsMove()</code>
Ramp_Mx_*_OF_Curr[]	uint16_t[]	Initialisiert aus <code>_OF[]</code> in <code>calcPulsePause()</code> ; in ISR dekrementiert
Current_Config	current_config_t	Gesetzt durch <code>calcPulsePause()</code> ; in ISR gelesen (<code>Minimal_Pulse</code>)

initGlobalsMove() und resetMoveTimers()

Diese zwei Funktionen werden sequenziell nach dem Ende jeder Bewegung aufgerufen:

```
// 1. initGlobalsMove() – vor resetMoveTimers():
Motor_Step_OF = 0; Motor_Step_OF_Curr = 0; // (alle 3 Achsen)
memset(Ramp_Mx_Rem_Pending, 1, ...);      // true für alle Einträge
Motor_Step_Corrector[] = 0;                // alle Korrektoren löschen
Ramp_Step_Curr = 0;                        // Rampen-Index zurücksetzen
```

```
// 2. resetMoveTimers():  
ftm0StopIRQ();    // alle CHIEs (ausser CH7) deaktivieren  
Motor_Step_Curr = Motor_Step_Max = 0; // (alle 3 Achsen + Rot)
```

Die Trennung in zwei Funktionen erlaubt, `initGlobalsRot()` separat zu rufen ohne die Linearachsen-Globals zu beeinflussen.

Erstellen & Flashen

Voraussetzungen

- MCUXpresso IDE (NXP) — empfohlen, enthält CMSIS und Kinetis SDK
- Oder: `arm-none-eabi-gcc` Toolchain mit GNU Make
- J-Link Debug-Probe **oder** OpenSDA (auf MC_CAR-Board integriert)
- Optional: J-Link Commander für Kommandozeilen-Flash

Erstellen mit MCUXpresso

1. Projekt importieren: **File** → **Import** → **Existing Projects into Workspace**
2. Das Verzeichnis `PREN_Project_Share` auswählen
3. Die Build-Konfiguration **Debug** wählen
4. Auf **Build** klicken (oder `Ctrl+B` drücken)

Das Ausgabe-Artefakt ist `Debug/PREN_Project_Share.axf`.



Bei Compiler-Warnings zu `implicit declaration` oder undeklarierten Funktionen: Sicherstellen, dass alle Header-Dateien korrekt eingebunden sind. Die Reihenfolge der Includes in `motor.c` und `calculation.c` ist relevant.

Erstellen über die Kommandozeile

```
cd Debug  
make -j4
```

Flashen & Debuggen

Debug-Probe anschliessen und die integrierte **Debug**-Startkonfiguration von MCUXpresso verwenden, oder über den J-Link Commander flashen:

```
JLinkExe -device MK22FN512xxx12 -if SWD -speed 4000  
> loadfile Debug/PREN_Project_Share.axf  
> r  
> go
```

Für automatisiertes Flashen (z.B. in CI):

```
JLinkExe -device MK22FN512xxx12 -if SWD -speed 4000 -autoconnect 1 \
```

```
-CommanderScript flash.jlink
```

mit `flash.jlink`:

```
loadfile Debug/PREN_Project_Share.axf
r
go
exit
```

Kompilierzeit-Konfiguration anpassen

Wichtige Flags in `globals.h` (Motortiming & Rampe):

```
// Timing
#define MOTOR_PULSE_US      200    // Pulsbreite [µs]
#define MIN_STEP_DISTANCE_US 200    // Mindestpause [µs]
#define SLOW_MODE_FACTOR    4      // Geschwindigkeits-Divisor für Pick&Place

// Rampe (Anlauf)
#define RAMP_MODE_NSTEP      1      // N-Schritt-Rampe aktivieren
#define RAMP_NSTEPS          10     // Anzahl Rampen-Stufen
#define RAMP_NSTEPS_STEPS    20000  // Ticks pro Stufe
#define RAMP_NSTEPS_STEP_PERC 100   // Prozentuale Verlängerung pro Stufe [%]

// Endrampe (Bremsen)
#define RAMP_MODE_END        1
#define RAMP_END_PS1         150    // Prescaler-Reduktion ab N verbleibenden
Schritten
#define RAMP_END_PS2         50
```

Wichtige Flags in `platform.h` (Hardware):

```
#define PLATFORM    AUTO    // Board-Erkennung zur Laufzeit (MC_CAR oder TinyK22)
#define DEBUG_LED    0      // Debug-LEDs aktivieren
#define TERMINAL_BAUDRATE 57600 // Debug-UART Baudrate
```

Debug-Modi

Folgende Flags in `globals.h` vor dem Erstellen setzen:

```
#define DEBUG_MODE      1    // Debug-Pin-Ausgabe (RES1), Hauptschleife deaktivieren
#define TEST_SEQUENCE    1    // Automatisierten Motortest beim Start ausführen
#define ISR_MONITOR      1    // Reserve-Pins in ISRs für Timing-Analyse umschalten
```

Mit `DEBUG_MODE=1` werden zusätzlich aktiviert (wenn 1 gesetzt):

```
#define DEBUG_MODE_ISR1 1 // RES1-Pin in CH1-ISR togglen (Motor 1 Timing)
#define DEBUG_MODE_ISR2 1 // RES1-Pin in CH2-ISR togglen (Motor 2 Timing)
#define DEBUG_MODE_ISR3 1 // RES1-Pin in CH4-ISR togglen (Motor 3 Timing)
#define DEBUG_MODE_SEQ 1 // CH6-Sequenz-Monitoring
```

Diese Pins können am Oszilloskop gemessen werden, um ISR-Dauer und Timing-Jitter zu analysieren.

Häufige Build-Probleme

Problem	Lösung
Linker-Fehler <code>undefined reference to Ramp_M1_...</code>	Sicherstellen, dass <code>control.c</code> mit dem Rest kompiliert wird — dort sind die globalen Ramp-Arrays definiert
FTM0 feuert nicht	Prüfen ob <code>ftm0StartClk()</code> aufgerufen wurde und <code>ftm0EnableIRQ()</code> vor dem Start aktiv ist
Motoren starten nicht synchron	<code>RAMP_NSTEPS_FIRST_MOD</code> zu klein — CnV-Wert < ISR-Ausführungszeit → CH6 feuert zu früh
Emergency-Stop löst unerwartet aus	CH7-CnV prüfen; bei Init muss ein gültiger Auslöse-Zeitpunkt gesetzt sein